

NextJS

Table of Contents

01-nextjs

02-app-router

03-routing

04-server-components

05-client-components

06-data-fetching

07-static-dynamic-rendering

08-client-data-fetching

09-api-routes

10-image-component

11-metadata

12-middleware

13-public-folder

temas-pendientes

Next.js

Next.js es un **framework de React** que agrega:

- **SSR** (Server Side Rendering)
- **SSG** (Static Site Generation)
- **Routing automático** basado en archivos
- **API backend** dentro del mismo proyecto
- **Optimización automática** de imágenes, fuentes y bundles

Next.js 14 — Lo más importante

- App Router
- Server Components
- Client Components
- Routing dinámico
- Layouts
- Fetch en server
- API routes
- Middleware
- Optimización de imágenes

Instalación

TEXT

```
cd carpeta-del-proyecto
npx create-next-app@latest
# o para una versión específica:
npx create-next-app@14
```

```
cd project-name
npm run dev
```

Setup prompts

TEXT

- ✓ Would you like to use TypeScript? ... No / Yes
- ✓ Would you like to use ESLint? ... No / Yes
- ✓ Would you like to use Tailwind CSS? ... No / Yes
- ✓ Would you like to use `src/` directory? ... No / Yes
- ✓ Would you like to use App Router? (recommended) ... No / Yes
- ✓ Would you like to customize the default import alias (@/*)? ... No / Yes

Comandos

Comando	Descripción
<code>npm run dev</code>	Iniciar servidor de desarrollo
<code>npm run build</code>	Compilar para producción
<code>npm start</code>	Iniciar servidor de producción
<code>npm run lint</code>	Ejecutar ESLint

Turbopack

Turbopack es el bundler incremental de Next.js, sucesor de Webpack. Está escrito en Rust y es significativamente más rápido. Se usa automáticamente con `npm run dev`.

Tailwind CSS

Next.js incluye soporte integrado para Tailwind CSS al crearlo con la opción correspondiente. Tailwind es un framework CSS de tipo utility-first que permite escribir estilos directamente en el HTML/JSX usando clases predefinidas.

TSX

```
<button className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4">
  Click me
</button>
```

Navegación rápida

Archivo	Contenido
App Router	Archivos reservados, estructura de carpetas, layouts, proyecto
Routing	Pages Router, App Router, Link, usePathname, useRouter, rutas dinámicas
Server Components	Componentes de servidor, ventajas, limitaciones
Client Components	Directiva 'use client', interactividad, hooks
Data Fetching	Fetch en server, async/await, caching, loading/error states
Static & Dynamic Rendering	SSG, ISR, SSR, cache strategies
Client Data Fetching	Fetch en cliente, useState/useEffect vs React Query, tradeoffs
API Routes	Route handlers, GET/POST, rutas dinámicas en API
Image Component	Optimización, lazy loading, remotePatterns
Metadata	Metadata API, generateMetadata, Open Graph
Middleware	Middleware, matcher, autenticación, redirecciones
Public Folder	Archivos estáticos, favicon, assets

App Router

El **App Router** es el sistema de enrutamiento moderno de Next.js 14, basado en la carpeta `app/`. Soporta Server Components, layouts anidados y nuevas formas de obtener datos.

Archivos Reservados

Archivo	Descripción
<code>page.tsx</code>	Define el contenido de la página para una ruta específica
<code>layout.tsx</code>	Layout que envuelve las páginas, persiste entre navegaciones
<code>loading.tsx</code>	UI de carga mientras se obtienen datos
<code>error.tsx</code>	UI de error (con botón reset)
<code>not-found.tsx</code>	UI para rutas no encontradas (404)

Estructura de Carpetas

Cada carpeta representa un segmento de la URL:

TEXT

```
app/
  page.tsx           → /
  layout.tsx         → Layout raíz (envuelve todo)
  products/
    page.tsx         → /products
    [id]/
      page.tsx       → /products/:id
```

La URL `/products/new` corresponde a la carpeta `products/new/` con un archivo `page.tsx` dentro.

Layouts

Un **layout** es un componente que envuelve páginas y persiste entre navegaciones (no se re-renderiza al cambiar de página).

TSX

```
// app/layout.tsx – Layout raíz
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="es">
      <body>
        <header>Header global</header>
        {children}
        <footer>Footer global</footer>
      </body>
    </html>
  );
}
```

Layouts anidados

TEXT

app/	
layout.tsx	→ Layout raíz (global)
shop/	
layout.tsx	→ Layout solo para /shop/*
page.tsx	→ /shop

TSX

```
// app/shop/layout.tsx
export default function ShopLayout({ children }: { children: React.ReactNode }) {
  return (
    <div className="shop-layout">
      <aside>Sidebar de la tienda</aside>
      <main>{children}</main>
    </div>
  );
}
```

Rutas Dinámicas

Crea una carpeta con `[param]` para rutas dinámicas. El parámetro se accede via `params` :

TEXT

```
app/  
  users/  
    page.tsx      → /users  
    [userId]/  
      page.tsx    → /users/:userId
```

TSX

```
// app/users/[userId]/page.tsx  
export default async function UserPage({ params }: { params: { userId: string } }) {  
  const user = await getUser(params.userId);  
  return <div>Usuario: {user.name}</div>;  
}
```

Estructura de proyecto

El App Router necesita una carpeta `app`, pero puede estar dentro de `src/`.

Sin `src/` (estructura plana)

TEXT

```
mi-proyecto/
├── app/
│   ├── favicon.ico
│   ├── globals.css
│   ├── layout.tsx
│   ├── page.tsx
│   ├── loading.tsx
│   ├── not-found.tsx
│   ├── robots.ts
│   ├── sitemap.ts
│   ├── contacto/
│   │   └── page.tsx
│   ├── precios/
│   │   └── page.tsx
│   └── api/
│       └── contacto/
│           └── route.ts
├── components/
├── hooks/
├── lib/
├── services/
├── utils/
├── public/
├── package.json
├── next.config.ts
├── tsconfig.json
└── .env
```

Con **src/** (recomendado)


```
mi-proyecto/  
├── src/  
│   ├── app/  
│   │   ├── favicon.ico  
│   │   ├── globals.css  
│   │   ├── layout.tsx  
│   │   ├── page.tsx  
│   │   ├── loading.tsx  
│   │   ├── not-found.tsx  
│   │   ├── robots.ts  
│   │   ├── sitemap.ts  
│   │   ├── contacto/  
│   │   │   └── page.tsx  
│   │   ├── precios/  
│   │   │   └── page.tsx  
│   │   └── api/  
│   │       └── contacto/  
│   │           └── route.ts  
│   ├── components/  
│   ├── features/  
│   ├── hooks/  
│   ├── lib/  
│   ├── services/  
│   ├── types/  
│   ├── utils/  
│   └── styles/  
├── public/  
├── package.json  
├── next.config.ts  
├── tsconfig.json  
└── .env
```

Global styles

Los estilos globales se definen en `globals.css` y se importan únicamente en el layout raíz (`app/layout.tsx`). No se pueden importar en otros componentes o layouts.

Routing

Next.js tiene **dos sistemas de enrutamiento**: Pages Router (legado) y App Router (nuevo, recomendado).

Pages Router (legado)

Sistema basado en archivos dentro de la carpeta `pages/` :

TEXT

```
pages/  
  index.js      → /  
  about.js      → /about  
  products/  
    index.js    → /products  
    [id].js     → /products/:id
```

App Router (recomendado)

Sistema basado en la carpeta `app/` con Server Components y layouts:

TEXT

```
app/  
  page.tsx      → /  
  layout.tsx     → Layout raíz  
  products/  
    page.tsx     → /products  
    [id]/  
      page.tsx   → /products/:id
```

Link Component

El componente `<Link>` reemplaza a la etiqueta `<a>` para navegación del lado del cliente sin recargar la página:

TSX

```
import Link from 'next/link';

export default function Navigation() {
  return (
    <nav>
      <Link href="/">Home</Link>
      <Link href="/products">Productos</Link>
      <Link href="/about">Sobre nosotros</Link>
    </nav>
  );
}
```

usePathname Hook

Devuelve la ruta actual de la URL. Solo funciona en Client Components:

TSX

```
'use client';

import { usePathname } from 'next/navigation';

export default function ActiveLink({ href, children }: { href: string; children: React.ReactNode }) {
  const pathname = usePathname();
  const isActive = pathname === href;

  return (
    <Link href={href} className={isActive ? 'active' : ''}>
      {children}
    </Link>
  );
}
```

useRouter Hook

Para navegación programática:

TSX

```
'use client';

import { useRouter } from 'next/navigation';

export default function LoginButton() {
  const router = useRouter();

  return (
    <button onClick={() => router.push('/dashboard')}>
      Ir al dashboard
    </button>
  );
}
```

useParams Hook

Para acceder a los parámetros de ruta en Client Components:

TSX

```
'use client';

import { useParams } from 'next/navigation';

export default function ProductDetail() {
  const params = useParams<{ id: string }>();
  return <div>Producto ID: {params.id}</div>;
}
```

Server Components

En Next.js 14, **todos los componentes son Server Components por defecto**. Se ejecutan en el servidor, no en el navegador.

Ventajas

- **Menos JavaScript** enviado al cliente
- **Carga de datos más rápida** — acceso directo a bases de datos, APIs y sistema de archivos
- **Mayor seguridad** — código sensible (tokens, queries) nunca llega al navegador

Ejemplo

TSX

```
// Este componente se ejecuta en el servidor
export default async function ProductList() {
  const products = await db.product.findMany();

  return (
    <ul>
      {products.map(product => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  );
}
```

Los Server Components pueden ser `async` y hacer fetch directamente sin necesidad de `useEffect`.

Limitaciones

- No pueden usar hooks (`useState`, `useEffect`, `useRouter`, etc.)
- No pueden manejar eventos del lado del cliente (`onClick`, `onSubmit`)
- No pueden acceder a APIs del navegador (`localStorage`, `window`, `document`)
- No pueden usar context que dependa de estado del cliente

Client Components

Cuando un componente necesita **interactividad** (hooks, eventos, APIs del navegador), debe marcarse como **Client Component** agregando `'use client'` al inicio del archivo.

Directiva 'use client'

TSX

```
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

¿Cuándo usar Client Components?

- Manejo de estado (useState, useReducer)
- Efectos secundarios (useEffect)
- Eventos del usuario (onClick, onChange, onSubmit)
- Hooks de navegación (usePathname, useRouter, useParams)
- APIs del navegador (localStorage, IntersectionObserver, window)
- Contextos que usan estado del lado del cliente

Buenas prácticas

Mantén los Client Components en las **hojas del árbol** de componentes. Un Server Component puede renderizar un Client Component, pero no al revés.

TEXT

app/	
page.tsx	→ Server Component (contiene datos)
ProductCard.tsx	→ Client Component (tiene interactividad)

Server Components dentro de Client Components

Puedes pasar Server Components como `children` a un Client Component:

TSX

```
// ClientComponent.tsx
'use client';
export default function ClientComponent({ children }: { children: React.ReactNode }) {
  const [isOpen, setIsOpen] = useState(false);
  return <div onClick={() => setIsOpen(!isOpen)}>{children}</div>;
}

// page.tsx (Server Component)
import ClientComponent from './ClientComponent';
import ServerProductList from './ServerProductList';

export default function Page() {
  return (
    <ClientComponent>
      <ServerProductList /> {/* Esto sigue siendo Server Component */}
    </ClientComponent>
  );
}
```

Data Fetching en Server Components

En Next.js 14, al usar Server Components, ya no necesitas `useEffect` para consumir APIs. Puedes crear funciones `async` fuera del componente y usarlas directamente dentro.

Patrón recomendado

TSX

```
// La función fetch se define fuera del componente
async function getPosts() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts')
  if (!res.ok) throw new Error('Error al obtener posts');
  return res.json();
}

// El componente es async y espera los datos
export default async function PostsPage() {
  const posts = await getPosts();

  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Data Fetching en Layouts

TSX

```
async function getCategories() {
  const res = await fetch('/api/categories');
  return res.json();
}

export default async function ShopLayout({ children }: { children: React.ReactNode }) {
  const categories = await getCategories();

  return (
    <div>
      <nav>
        {categories.map(cat => (
          <Link key={cat.id} href={` /shop/${cat.slug}`} >{cat.name}
        ))}
      </nav>
      {children}
    </div>
  );
}
```

Caching y Revalidación

Por defecto, `fetch` en Server Components cachea los resultados automáticamente.

TSX

```
// Sin cache – siempre datos frescos
fetch(url, { cache: 'no-store' });

// Revalidación por tiempo (segundos)
fetch(url, { next: { revalidate: 60 } });

// Tag para revalidación manual
fetch(url, { next: { tags: ['products'] } });
```

Loading y Error States

Crea archivos en la misma carpeta de la ruta para manejar estados automáticamente:

TEXT

products/	
page.tsx	→ Contenido de la página
loading.tsx	→ UI mientras carga (Suspense automático)
error.tsx	→ UI cuando hay error

TSX

```
// loading.tsx
export default function Loading() {
  return <div>Cargando productos... </div>;
}

// error.tsx
'use client';
export default function Error({ error, reset }: { error: Error; reset: () => void }) {
  return (
    <div>
      <h2>Error al cargar</h2>
      <button onClick={reset}>Reintentar</button>
    </div>
  );
}
```

Fetch paralelo

TSX

```
export default async function Page() {
  // Las promesas se ejecutan en paralelo
  const [products, categories] = await Promise.all([
    getProducts(),
    getCategories(),
  ]);

  return <div>... </div>;
}
```

Static y Dynamic Rendering

Next.js ofrece diferentes estrategias de renderizado según cómo y cuándo se genera el HTML.

Static Rendering (SSG)

El HTML se genera **en build time**. Es la opción por defecto cuando haces fetch sin opciones especiales.

TSX

```
// Static – se genera al hacer build
async function getPosts() {
  const res = await fetch('https://api.example.com/posts');
  return res.json();
}
```

Características

- HTML generado una sola vez en tiempo de build
- Resultado cacheado y reutilizado en todas las peticiones
- Ideal para contenido que no cambia (blog, docs, landing pages)
- Máximo rendimiento y SEO

Revalidación con ISR

Puedes mantener Static Rendering pero re-generar la página cada cierto tiempo:

TSX

```
// ISR – Static pero se revalida cada 60 segundos
fetch(url, { next: { revalidate: 60 } });
```

Dynamic Rendering (SSR)

El HTML se genera **en cada request**. Se activa automáticamente cuando usas:

- `fetch(url, { cache: 'no-store' })`
- `headers()` o `cookies()` en un Server Component
- `dynamic = 'force-dynamic'` en el layout/page

TSX

```
// Dynamic – se genera en cada petición
async function getPosts() {
  const res = await fetch('https://api.example.com/posts', {
    cache: 'no-store',
  });
  return res.json();
}
```

Características

- HTML generado por cada request
- Datos siempre frescos
- Mayor latencia que Static
- Ideal para contenido personalizado o que cambia frecuentemente

Resumen

Estrategia	Método	Cuándo usarlo
Static (SSG)	<code>fetch(url)</code> (por defecto)	Contenido que no cambia
ISR	<code>fetch(url, { next: { revalidate: N } })</code>	Contenido que cambia ocasionalmente
Dynamic (SSR)	<code>fetch(url, { cache: 'no-store' })</code>	Contenido que cambia en cada request

Data Fetching en el Cliente

Cuando necesitas obtener datos del lado del cliente (en un Client Component), tienes dos opciones principales.

useState + useEffect

TSX

```
'use client';

import { useState, useEffect } from 'react';

export default function PostsList() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/posts')
      .then(res => res.json())
      .then(data => {
        setPosts(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <div>Cargando... </div>;

  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

React Query (TanStack Query)

Alternativa más robusta que el manejo manual con hooks. Ofrece caché, revalidación, refetch automático, y menos boilerplate.

TSX

```
'use client';

import { useQuery } from '@tanstack/react-query';

export default function PostsList() {
  const { data: posts, isLoading } = useQuery({
    queryKey: ['posts'],
    queryFn: () => fetch('/api/posts').then(res => res.json()),
  });

  if (isLoading) return <div>Cargando...</div>;

  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Desventajas del fetching en cliente

Problema	Descripción
Bundles grandes	El código de fetching y las librerías (React Query, etc.) se envían al navegador
Resource intensive	El cliente hace peticiones HTTP, consume batería y datos móviles
No SEO	Los datos no están disponibles en el HTML inicial (renderizado en blanco hasta que JS carga y fetch termina)
API keys expuestas	Cualquier secreto en el cliente es visible en las DevTools

Recomendación

Siempre que sea posible, usa **Server Components** para fetching. El fetching en cliente solo cuando no hay alternativa (eventos del usuario, polling en tiempo real, datos que cambian sin recargar página).

API Routes (Route Handlers)

En Next.js 14 puedes crear **rutas de API** dentro de tu proyecto usando el App Router. Se definen con un archivo `route.ts` dentro de una carpeta.

Estructura

TEXT

```
app/  
  api/  
    products/  
      route.ts      → GET /api/products  
    products/  
      [id]/  
        route.ts    → GET /api/products/:id
```

Métodos HTTP

TSX

```
// app/api/products/route.ts  
import { NextResponse } from 'next/server';  
  
export async function GET() {  
  const products = await db.product.findMany();  
  return NextResponse.json(products);  
}  
  
export async function POST(request: Request) {  
  const body = await request.json();  
  const product = await db.product.create({ data: body });  
  return NextResponse.json(product, { status: 201 });  
}
```

Rutas dinámicas en API

TSX

```
// app/api/products/[id]/route.ts
import { NextResponse } from 'next/server';

export async function GET(
  _request: Request,
  { params }: { params: { id: string } }
) {
  const product = await db.product.findUnique({
    where: { id: params.id }
  });
  if (!product) {
    return NextResponse.json({ error: 'Not found' }, { status: 404 })
  }
  return NextResponse.json(product);
}

export async function PUT(
  request: Request,
  { params }: { params: { id: string } }
) {
  const body = await request.json();
  const product = await db.product.update({
    where: { id: params.id },
    data: body
  });
  return NextResponse.json(product);
}

export async function DELETE(
  _request: Request,
  { params }: { params: { id: string } }
) {
  await db.product.delete({ where: { id: params.id } });
  return NextResponse.json({ message: 'Producto eliminado' });
}
```

Request y Response

Next.js usa las APIs estándar de Web (`Request` y `Response`) junto con `NextRequest` y `NextResponse` que agregan utilidades como cookies, headers y redirects.

TSX

```
import { NextRequest, NextResponse } from 'next/server';

export async function GET(request: NextRequest) {
  const searchParams = request.nextUrl.searchParams;
  const query = searchParams.get('q');

  const token = request.cookies.get('token');

  return NextResponse.json({ query, authenticated: !!token });
}
```

Revalidación desde API Routes

TSX

```
import { revalidateTag } from 'next/cache';

export async function POST(request: Request) {
  const body = await request.json();
  const product = await createProduct(body);
  revalidateTag('products'); // Invalida cache de fetch con ese tag
  return NextResponse.json(product, { status: 201 });
}
```

Image Component

Next.js provee el componente `<Image>` que **optimiza imágenes automáticamente**: compresión, lazy loading, formatos modernos (WebP/AVIF) y soporte responsive.

Uso básico

TSX

```
import Image from 'next/image';

export default function Page() {
  return (
    <Image
      src="/hero.jpg"
      alt="Hero image"
      width={1200}
      height={600}
    />
  );
}
```

Características

- **Lazy loading** por defecto — solo carga cuando la imagen entra en el viewport
- **Compresión automática** y conversión a WebP/AVIF
- **Aspect ratio correcto** — se define con `width` y `height`
- **Carga extremadamente rápida** gracias a la optimización en build time

Atributos importantes

Atributo	Descripción
<code>src</code>	Ruta de la imagen (local en <code>/public</code> o URL externa)
<code>alt</code>	Texto alternativo (obligatorio)
<code>width</code> / <code>height</code>	Dimensiones para aspect ratio correcto
<code>sizes</code>	Define tamaños para diferentes viewports
<code>priority</code>	Prioriza carga (above the fold, desactiva lazy loading)
<code>placeholder="blur"</code>	Muestra un placeholder borroso mientras carga
<code>className</code>	Clases CSS para estilizar

TSX

```
<Image
  src="/hero.jpg"
  alt="Hero"
  sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
  priority
  placeholder="blur"
/>
```

Imágenes externas

Para usar imágenes de dominios externos, debes configurarlos en `next.config.ts` :

TS

```
// next.config.ts
import type { NextConfig } from 'next';

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      { hostname: 'upload.wikipedia.org' },
      { hostname: 'images.unsplash.com' },
    ],
  },
};

export default nextConfig;
```

Sin esta configuración, Next.js bloqueará las imágenes de orígenes externos por seguridad.

Imágenes locales (import)

Para imágenes locales, Next.js detecta automáticamente las dimensiones:

TSX

```
import logo from './logo.png';

export default function Header() {
  return <Image src={logo} alt="Logo" placeholder="blur" />;
}
```

Metadata

Next.js tiene una API integrada para manejar **metadatos** (title, description, Open Graph, etc.) sin necesidad de librerías externas. Solo funciona en Server Components.

Metadata estática

TSX

```
export const metadata = {
  title: 'Mi Página',
  description: 'Descripción de mi página',
};
```

Metadata dinámica

TSX

```
export async function generateMetadata({ params }: { params: { id: string } }) {
  const product = await getProduct(params.id);
  return {
    title: product.name,
    description: product.description,
  };
}
```

Open Graph y más

TSX

```
export const metadata = {
  title: 'Mi Página',
  description: 'Descripción',
  openGraph: {
    title: 'Mi Página en redes',
    description: 'Descripción para redes sociales',
    images: ['/og-image.png'],
  },
  twitter: {
    card: 'summary_large_image',
  },
};
```

Metadata por ruta

Cada `page.tsx` puede tener su propia metadata. Los valores definidos en el layout padre sirven como **fallback** para las páginas hijas. Si la página define un campo, sobrescribe al del layout.

TSX

```
// app/layout.tsx – valores por defecto
export const metadata = {
  title: 'Mi Sitio',
  description: 'Descripción global',
};

// app/products/page.tsx – sobrescribe title, hereda description
export const metadata = {
  title: 'Productos',
};
```

Para títulos con formato, usa `title.template` en el layout:

TSX

```
// app/layout.tsx
export const metadata = {
  title: { default: 'Mi Sitio', template: '%s | Mi Sitio' },
};

// app/products/page.tsx → renderiza "Productos | Mi Sitio"
export const metadata = {
  title: 'Productos',
};
```

Middleware

El **Middleware** en Next.js te permite ejecutar código antes de que se complete una petición. Es ideal para autenticación, redirecciones, headers, etc.

Archivo

Se coloca en la raíz del proyecto como `middleware.ts` :

TS

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  // Ejemplo: redirigir a login si no hay token
  const token = request.cookies.get('token');
  if (!token) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/profile/:path*'],
};
```

Cuándo usarlo

- **Autenticación** — proteger rutas, redirigir a login
- **Redirecciones** — por idioma, país, dispositivo
- **Headers** — modificar request/response headers
- **Geolocalización** — detectar país del usuario (Vercel)
- **A/B testing** — redirigir a variantes
- **Rate limiting** — controlar frecuencia de peticiones

Matcher

El `matcher` define qué rutas ejecutan el middleware. Usa patrones de ruta:

TS

```
export const config = {  
  matcher: [  
    '/dashboard/:path*',  
    '/admin/:path*',  
    '/((?!api|_next/static|favicon.ico).*)', // excluir archivos está  
  ],  
};
```

El Middleware se ejecuta **antes** de que la petición llegue a la ruta, por lo que es muy eficiente y no afecta el rendimiento del renderizado.

Public Folder

La carpeta `public/` contiene **archivos estáticos** que se sirven directamente en la raíz del sitio.

Favicon

Si colocas un archivo `favicon.ico` en `public/`, Next.js lo detecta automáticamente y lo sirve sin configuración adicional.

Uso

Los archivos en `public/` se referencian desde la raíz (`/`):

TEXT

```
public/
  images/
    logo.png
  favicon.ico
  robots.txt
  sitemap.xml
  data/
    products.json
```

TSX

```
// Sin importar, solo la ruta desde /


// O con Image component
import Image from 'next/image';
<Image src="/images/logo.png" alt="Logo" width={200} height={100} />
```

Archivos comunes

- `favicon.ico` / `favicon.svg` — icono del sitio (auto-detectado)
- `robots.txt` — configuración para crawlers de buscadores
- `sitemap.xml` — mapa del sitio para SEO
- `og-image.png` — imagen para Open Graph en redes sociales
- `fonts/` — fuentes auto-hospedadas
- `data/` — datos JSON estáticos (ej. generados en build)

Temas pendientes — Next.js

- Server Actions — Mutaciones en server components sin API routes
- generateStaticParams — SSG con rutas dinámicas
- Route Groups — (auth), (shop) para organizar sin afectar URL
- Font Optimization — next/font, Google Fonts auto-hospedadas
- Environment Variables — NEXT_PUBLIC_, .env.local, validación
- Parallel Routes — @slots para múltiples vistas en una ruta
- Intercepting Routes — (.) para modales manteniendo contexto
- Deployment — Vercel, self-hosted, Docker